MiniSploit:

A CVE Scanner & Proof of Concept Tool

Mickaela L.

MacEwan University

CMPT 496: Final Project

April 22, 2026

1 Introduction

MiniSploit is a vulnerability scanning and validation system designed to identify software vulnerabilities on Ubuntu systems using publicly available CVE data. The system integrates automated vulnerability detection, proof-of-concept (PoC) exploit validation, and structured report generation into a single workflow.

MiniSploit operates using a client-server architecture, where a client system collects information on installed package versions and submits it to a server for analysis against a local CVE database.

The goal of this project is to demonstrate how vulnerabilities can be identified, validated, and communicated through a unified and transparent environment.

2 Background and Existing Solutions

Software systems are constantly exposed to newly discovered vulnerabilities, making vulnerability assessment and mitigation a core component of cybersecurity practice.

Vulnerabilities pose a direct risk to the confidentiality, integrity, and availability of systems by potentially exposing sensitive data, enabling unauthorized access or modification, and disrupting services. As a result, vulnerability scanning is a critical component of modern cybersecurity practices, allowing organizations to identify and address weaknesses before they can be exploited.

Modern vulnerability management relies on a combination of data sources, automated scanning tools, and validation techniques to identify and assess system vulnerabilities.

2.1 CVE Databases and Vulnerability Sources

The National Vulnerability Database (NVD) provides standardized vulnerability management data, including severity scores and machine-readable formats for automation.

The Snyk Vulnerability Database provides detailed package-level vulnerability records, including affected versions and remediation guidance.

2.2 Vulnerability Scanning Tools

Tools such as Nessus and OpenVAS demonstrate automated vulnerability detection and structured reporting. These tools analyze system configurations and installed software to identify known vulnerabilities.

2.3 Exploitation Frameworks

Frameworks such as Metasploit and repositories like Exploit-DB provide insight into how vulnerabilities are validated through exploitation. These tools allow security professionals to demonstrate the real-world impact of vulnerabilities beyond detection.

Each of these components informs a part of MiniSploit's design, combining vulnerability data, automated detection, and exploit validation within a unified system.

3 System Overview

MiniSploit follows a client-server architecture:

- The client collects system information, including installed packages and Ubuntu release version.
- The server processes this information and matches it against a local CVE database.
- The matching vulnerabilities are returned to the client.
- Optional proof-of-concept exploits can be executed.
- A structured security report is generated.

4 Data Sources

MiniSploit uses the Ubuntu CVE Tracker as its primary data source, which contains detailed information about vulnerabilities, affected packages, and patch status. This resource was selected because it offers distribution-specific data, allowing MiniSploit to accurately match vulnerabilities to Ubuntu package versions and releases.

Proof-of-concept exploits are sourced from publicly available repositories and are used to validate identified vulnerabilities in a controlled manner.

5 System Design and Implementation

5.1 Docker Environment

Docker is used to create controlled testing environments for demonstrating the MiniSploit system across different Ubuntu releases, including Focal (20.04), Jammy (22.04), and Noble (24.04).

These environments are intentionally configured with outdated packages to simulate real-world vulnerable systems. By using containers, testing can be performed safely and reproducibly without affecting the host system.

5.2 Server Implementation

The MiniSploit server is built using FastAPI and is responsible for processing client requests and performing vulnerability analysis.

When the server starts, it checks for a local copy of the Ubuntu CVE Tracker. If the data is missing or outdated, it updates the repository and rebuilds a local SQLite database containing parsed CVE information.

The server exposes an API endpoint (/scan) that accepts client data, processes package information, and returns matching vulnerabilities in JSON format. This process involves comparing client package data against the local CVE database by matching package names and evaluating version information. Vulnerabilities are identified by comparing the installed package version against the fixed version specified in the CVE data. A package is considered vulnerable if the installed version is lower than the patched version.

In addition to vulnerability scanning, the server provides a second endpoint (/run-poc) to support proof-of-concept execution. This endpoint enables controlled remote execution of exploit-related functionality for vulnerabilities that require remote interaction.

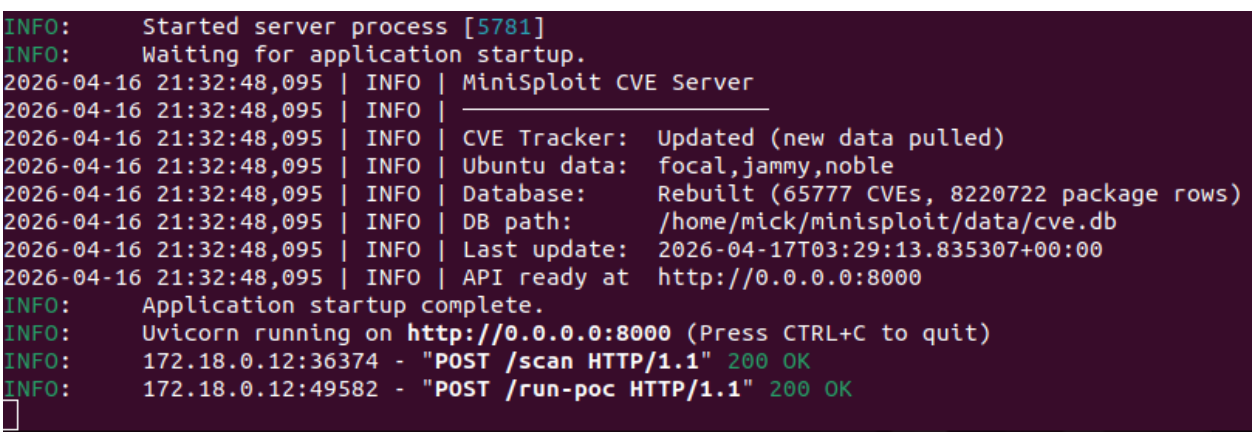

Figure 1: Server running and processing client requests

5.3 Client Implementation

The MiniSploit client runs on a vulnerable Ubuntu system.

Its responsibilities include:

- Collecting installed package information
- Sending data to the server
- Receiving vulnerability findings
- Saving results locally
- Prompting the user for permission to run PoC exploits
- Generating a structured HTML report

```
Sending inventory for 4ee713d3ebc8 (noble)
Server: http://192.168.56.104:8000/scan
Packages: 185

Findings by priority: {'high': 4, 'medium': 6, 'low': 5}

Findings returned: 15
- CVE-2026-24061 | inetutils 2:1.9.4-11ubuntu0.2 < 2:2.5-3ubuntu4.1 | high
- CVE-2025-32462 | sudo 1.9.15p5-3ubuntu5 < 1.9.15p5-3ubuntu5.24.04.1 | high
- CVE-2025-32463 | sudo 1.9.15p5-3ubuntu5 < 1.9.15p5-3ubuntu5.24.04.1 | high
- CVE-2026-35535 | sudo 1.9.15p5-3ubuntu5 < 1.9.15p5-3ubuntu5.24.04.2 | high
- CVE-2020-10188 | inetutils 2:1.9.4-11ubuntu0.2 < 2:1.9.4-12 | medium
- CVE-2021-40491 | inetutils 2:1.9.4-11ubuntu0.2 < 2:2.2-1 | medium
- CVE-2022-39028 | inetutils 2:1.9.4-11ubuntu0.2 < 2:2.3-5 | medium
- CVE-2023-40303 | inetutils 2:1.9.4-11ubuntu0.2 < 2:2.4-3ubuntu1 | medium
- CVE-2025-13151 | libtasn1-6 4.19.0-3ubuntu0.24.04.1 < 4.19.0-3ubuntu0.24.04.2 | medium
- CVE-2026-31790 | openssl 3.0.13-0ubuntu3.7 < 3.0.13-0ubuntu3.9 | medium
- CVE-2026-28387 | openssl 3.0.13-0ubuntu3.7 < 3.0.13-0ubuntu3.9 | low
- CVE-2026-28388 | openssl 3.0.13-0ubuntu3.7 < 3.0.13-0ubuntu3.9 | low
- CVE-2026-28389 | openssl 3.0.13-0ubuntu3.7 < 3.0.13-0ubuntu3.9 | low
- CVE-2026-28390 | openssl 3.0.13-0ubuntu3.7 < 3.0.13-0ubuntu3.9 | low
- CVE-2026-31789 | openssl 3.0.13-0ubuntu3.7 < 3.0.13-0ubuntu3.9 | low

Available Proof of Concept Exploits:
  [1] CVE-2026-24061.py (CVE: CVE-2026-24061, package: inetutils, priority: high)
  [2] CVE-2025-32462.sh (CVE: CVE-2025-32462, package: sudo, priority: high)
  [3] CVE-2025-32463.sh (CVE: CVE-2025-32463, package: sudo, priority: high)

To run one of these PoC Exploits, enter the number of the PoC you'd like to run.
If you'd like to run multiple, enter the numbers separated by commas, or 'all' to run all matches.
Otherwise, just press Enter to skip: all

Running PoC: CVE-2026-24061.py

Running PoC: CVE-2025-32462.sh

Running PoC: CVE-2025-32463.sh

Saved scan to: /client/scans/scan_4ee713d3ebc8_noble.json
Wrote report: /client/reports/report_4ee713d3ebc8_noble_20260417_034659.html
```

Figure 2: Client scanning system and detecting vulnerabilities

5.4 Security Report Generation

The report generation component transforms raw JSON scan results into a structured HTML document, presenting detected vulnerabilities, severity classifications, and detailed package

information in a clear and readable format.

MiniSploit Security Report

Risk Level: High

Client: `4ee713d3ebc8`
Ubuntu release: `noble`
Packages scanned: `185`
Affected packages: `4`
Vulnerabilities: `15`
Generated: `2026-04-17 03:46:59`

Vulnerabilities

Summary: High: 4 Medium: 6 Low: 5

Priority	CVE	Package	Installed	Fixed Version	Recommended Fix
High	CVE-2026-24061	inetutils	2:1.9.4-11ubuntu0.2	2:2.5-3ubuntu4.1	Upgrade inetutils to at least 2:2.5-3ubuntu4.1.
High	CVE-2025-32462	sudo	1.9.15p5-3ubuntu5	1.9.15p5-3ubuntu5.24.04.1	Upgrade sudo to at least 1.9.15p5-3ubuntu5.24.04.1.
High	CVE-2025-32463	sudo	1.9.15p5-3ubuntu5	1.9.15p5-3ubuntu5.24.04.1	Upgrade sudo to at least 1.9.15p5-3ubuntu5.24.04.1.

Figure 3: Generated security report with vulnerability details

5.4.1 Proof-of-Concept (PoC) Exploit Execution

To further validate detected vulnerabilities and demonstrate their potential impact, MiniSploit supports proof-of-concept (PoC) exploit execution.

When PoC exploits are executed, their output is incorporated into the report, providing additional context and evidence of vulnerability impact. This allows the report to function not only as a summary of findings, but also as documentation of validation results.

CVE: `CVE-2025-32462`
Package: `sudo`

Command Input

```
> id
> whoami
> hostname -I
```

Command Output

```
[+] Testing for CVE-2025-32462 bypass via 'sudo -h'...
[+] Success: Root access granted!
uid=0(root) gid=0(root) groups=0(root)
root
172.18.0.12
```

Exploit Execution Log

```
(empty)
```

Figure 4: Proof-of-concept exploit execution output

6 Results

The MiniSploit system identifies vulnerabilities based on installed package data and Ubuntu release versions, with the server accurately matching CVEs using the locally maintained database.

Proof-of-concept execution confirms that detected vulnerabilities can be validated within the testing environment. The generated reports clearly present vulnerability details, including severity classifications and, where applicable, exploit output.

7 Discussion

7.1 Challenges

The development of MiniSploit presented several challenges.

One challenge was creating reproducible vulnerable environments for testing. Initial at-

tempts using virtual machines proved difficult due to the lack of readily available vulnerable images. As a result, Docker containers were used instead, providing a more flexible and easily reproducible way of configuring vulnerable systems.

Another challenge involved comparing package names and versions between the client system and CVE data. Differences in naming conventions between the CVE database and the container caused missed matches, while non-standard version formats required additional handling to ensure accurate package version comparison.

7.2 Limitations

Limitations of the project include a limited proof-of-concept library and a focus on Ubuntu-based systems.

The system also does not currently implement authentication or input validation for API endpoints, making it unsuitable for use in untrusted environments.

7.3 Future Work

Future improvements could extend the functionality and robustness of the MiniSploit system as well as strengthen the security of the system:

- Expanding the proof-of-concept exploit library
- Improving CVE matching accuracy
- Including input validation and safeguards
- Adding authentication and secure communication
- Adding support for additional operating systems

8 Conclusion

MiniSploit demonstrates a complete vulnerability scanning workflow, from data collection to vulnerability detection, validation, and reporting. The project provides insight into real-world security practices and highlights the importance of structured vulnerability management systems. Future improvements could further enhance its applicability, particularly in areas such as security, scalability, and deployment.

9 References

- National Vulnerability Database (NVD): `https://nvd.nist.gov`

- Ubuntu CVE Tracker:
  - `https://ubuntu.com/security/cves`
  - `https://github.com/kees/ubuntu-cve-tracker`
- Exploit Database: `https://www.exploit-db.com`
- Metasploit Framework: `https://www.metasploit.com`
- OpenVAS: `https://www.openvas.org`